

CS121 Data Structures in C

09 *AVL Trees*

Outline

- What's AVL tree?
- Rebalance after insertion
 - ♦ Single rotation
 - ♦ Double rotation
- Implementation

Binary Search Tree

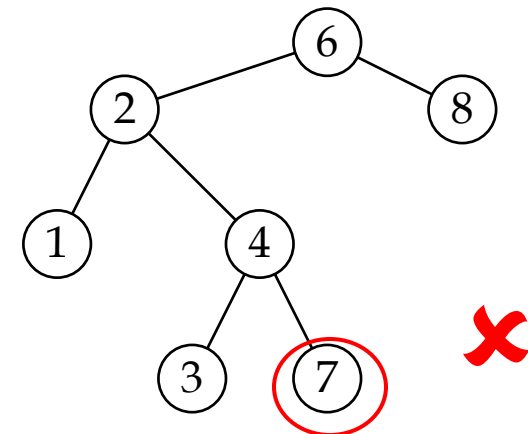
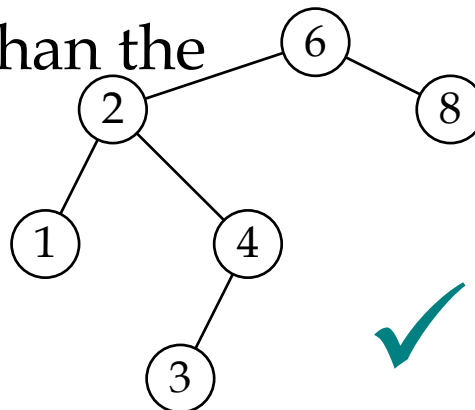
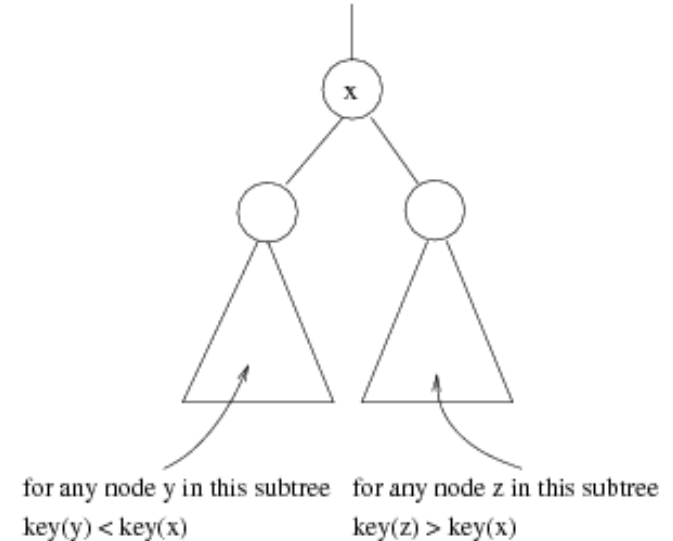
- A data structure for efficient search, insertion and deletion.

- Property of binary search tree

For every node X ,

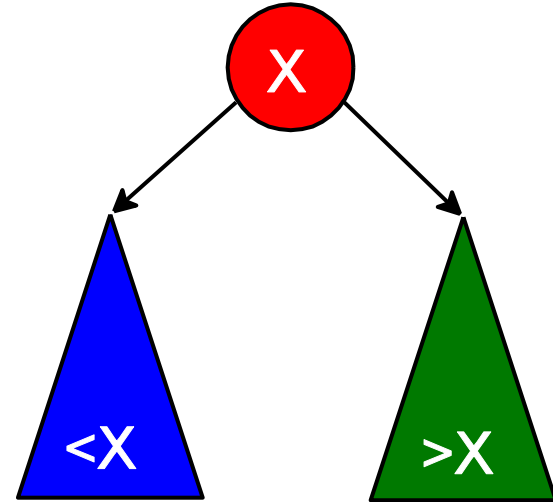
- ♦ all keys in its **left subtree** are **smaller** than the key value in X ; and
- ♦ all keys in its **right subtree** are **larger** than the key value in X .

And each subtree is itself a binary search tree.



Find Key X

- To look for a node with a certain Key
- $\text{Left} < \text{Node} < \text{Right}$
- Recursively do:
 1. Compare (Key,node)
 2. $\text{Key} < \text{node}$:
Go left
 3. $\text{Key} > \text{node}$:
Go right
- Time complexity = $O(\text{height})$



Question

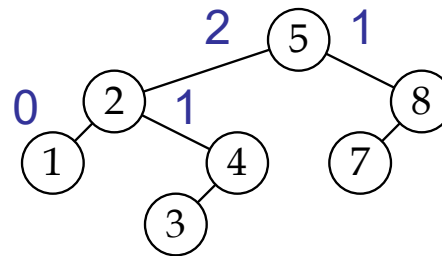
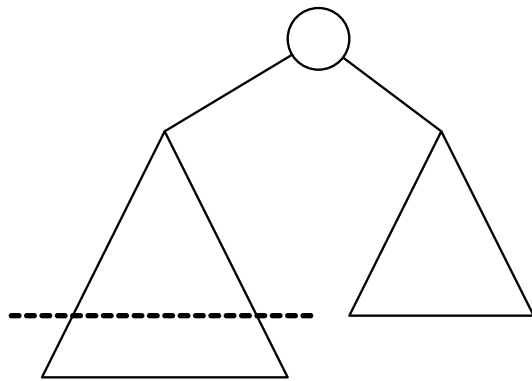
Let n be the size of a binary search tree. How can we keep its height $O(\log n)$ under insertion and deletion?

Perfectly Balanced Tree

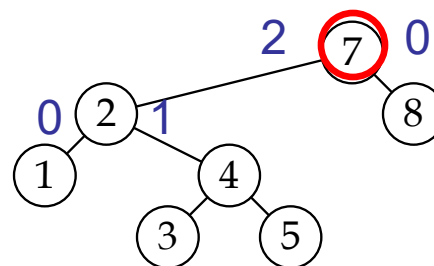
- A perfectly balanced tree requires that for every node, the sizes of its left and right sub-trees differ by at most 1.
- The balance condition is too rigid. It's hard to maintain a perfectly balanced search tree.
- We may relax the balance condition a little bit.
- It is the *depth* that actually affects the search time.

AVL Tree

- An *AVL tree* (Adelson-Velskii and Landis) is a *binary search tree* with a balanced condition
 - ♦ In which for *every* node in the tree, the height of the left and right subtrees differ by at most 1.



AVL tree

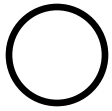


The AVL property is violated here

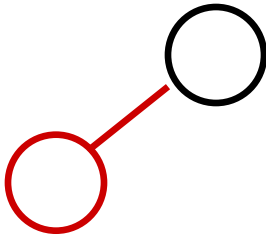
- ♦ Exercise: find the subtrees' heights of these nodes

AVL Tree with Minimum Number of Nodes

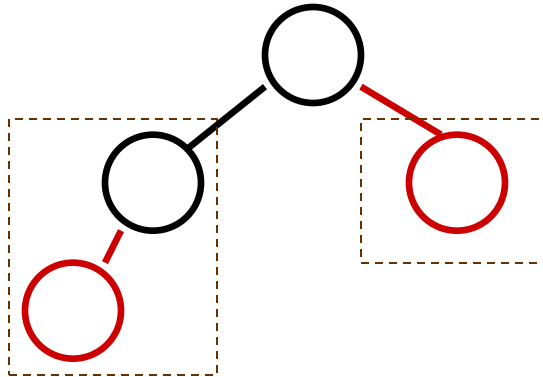
Let N_h be the minimum number of nodes in an AVL tree of height h .



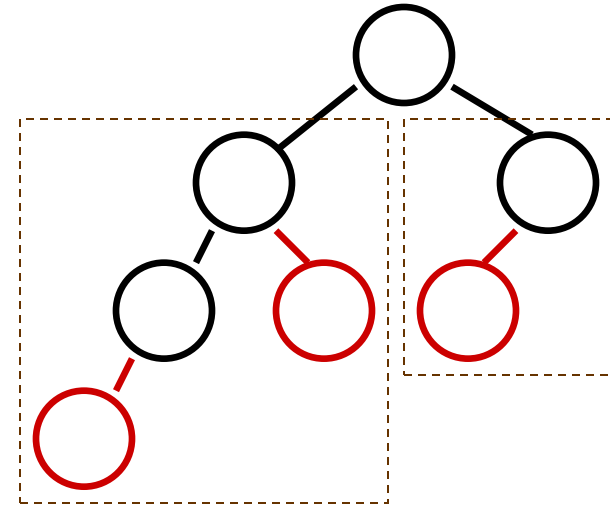
$$N_0 = 1$$



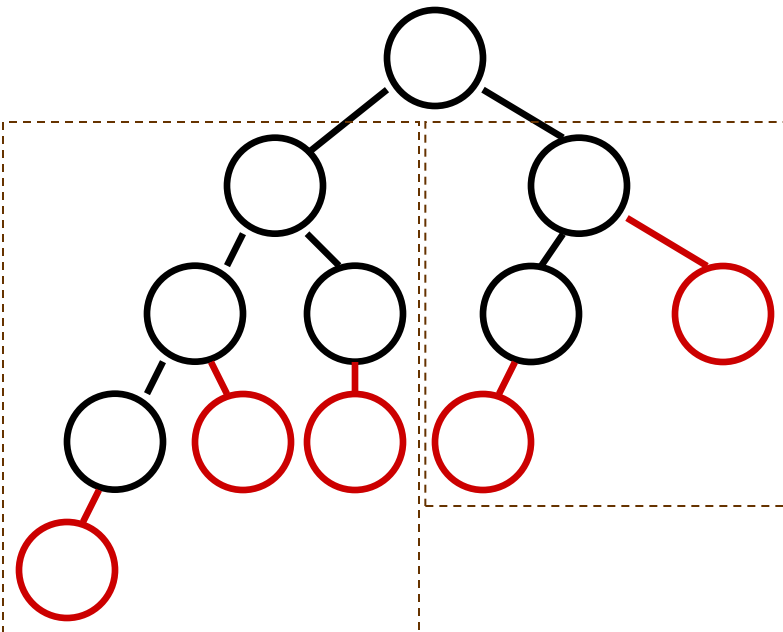
$$N_1 = 2$$



$$N_2 = 4$$



$$N_3 = N_1 + N_2 + 1 = 7$$



$$N_4 = N_3 + N_2 + 1 = 12$$

$$N(h) = N(h-1) + N(h-2) + 1,$$

$$N(0) = 1, N(1) = 2.$$

$N(h)$ is closely related to the Fibonacci numbers.

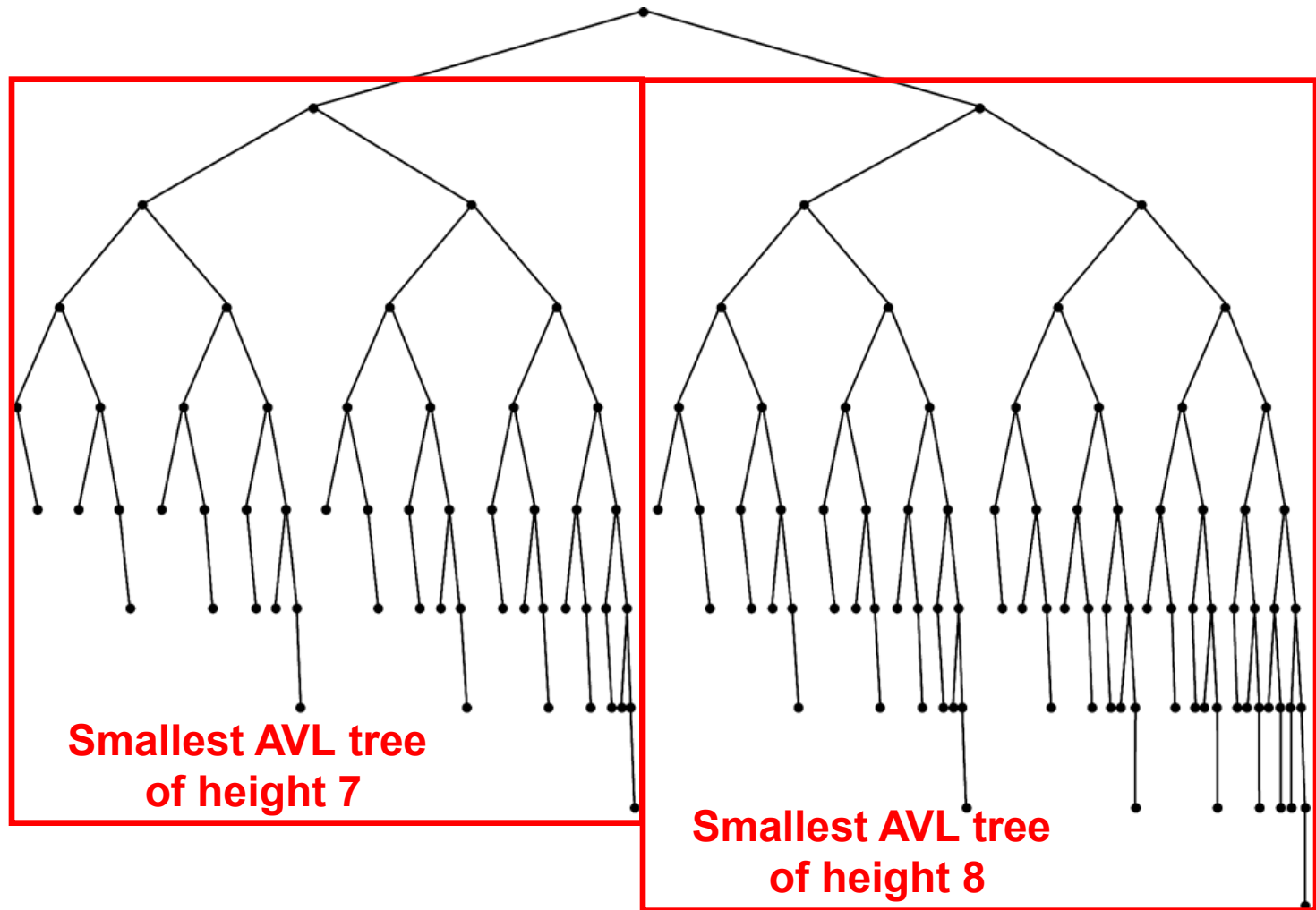
AVL Tree with Minimum Number of Nodes

- Let n_h denote the minimum number of nodes in an AVL tree of height h
- $n_0 = 1, n_1 = 2$ (base)
- $n_h = n_{h-1} + n_{h-2} + 1$
- For any AVL tree of height h and size n ,

$$n \geq n_h = n_{h-1} + n_{h-2} + 1 > 2n_{h-2} > 4n_{h-4} > \cdots > 2^i n_{h-2i}$$

- If h is even, let $i = h/2$. $\Rightarrow n > 2^{h/2} n_0 = 2^{h/2} \cdot 1 \Rightarrow h = O(\log n)$
- If h is odd, let $i = (h-1)/2$. $\Rightarrow n > 2^{(h-1)/2} n_1 = 2^{(h-1)/2} \cdot 2 \Rightarrow h = O(\log n)$
- Thus, many operations (e.g., insertion, deletion, and search) on an AVL tree will take $O(\log n)$ time

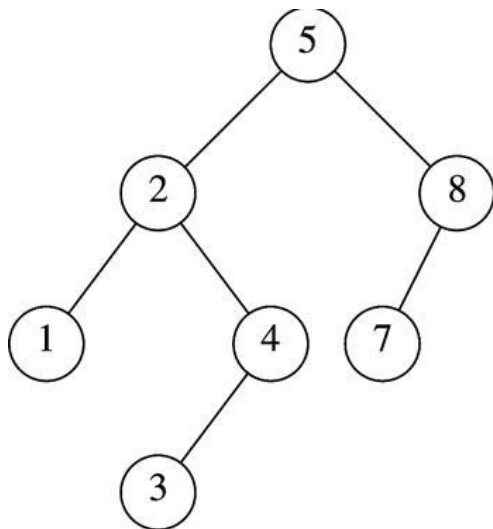
AVL Tree with Minimum Number of Nodes



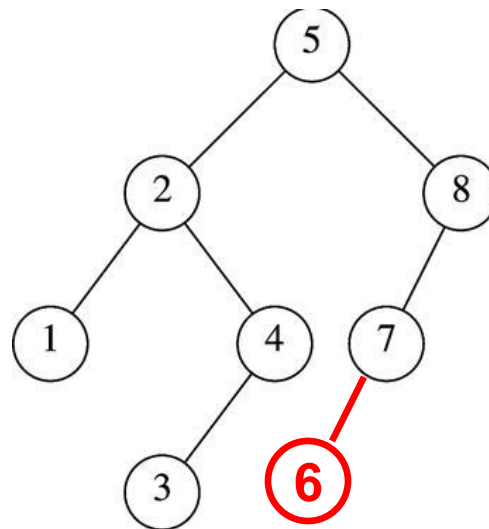
Smallest AVL tree of height 9

Insertion in AVL Trees

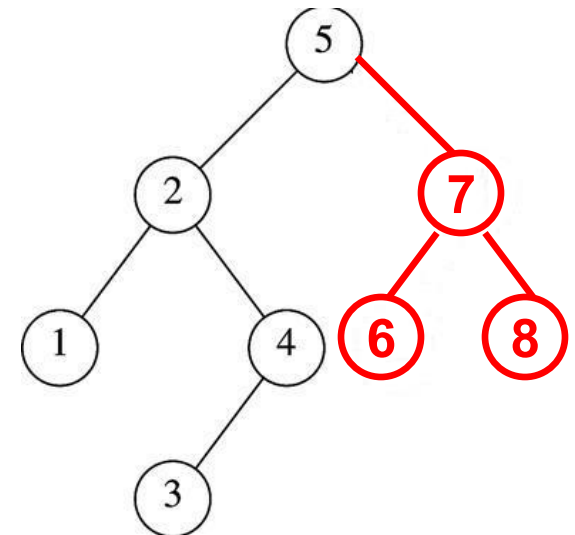
- Basically follows insertion strategy of binary search tree.
 - ♦ But it may cause violation of AVL tree property.
 - ♦ Restore the destroyed balance condition if needed.



Original AVL tree



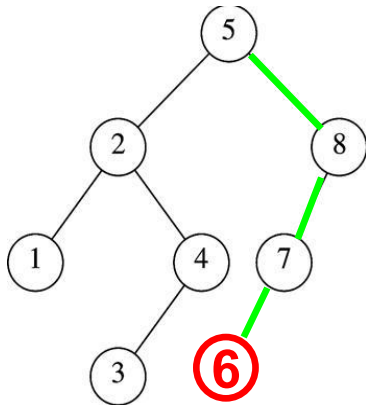
**Insert 6
Property violated**



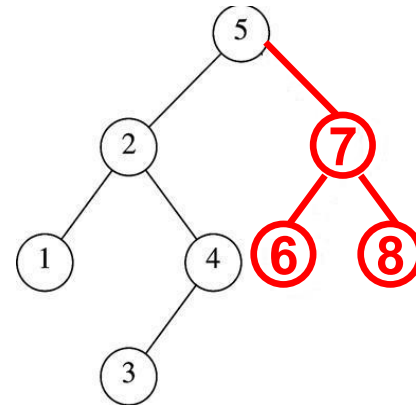
Restore AVL property

Some Observations of Violation

- After an insertion, only nodes that are on the path **from the insertion point to the root might** have their balance altered.
 - ♦ Because only those nodes have their subtrees altered.
- Rebalancing the tree at the deepest such node guarantees that the entire tree satisfies the AVL property.



Node 5,8,7 might
have balance altered



Rebalancing node 7
guarantees the whole tree be AVL

Tree Rotations

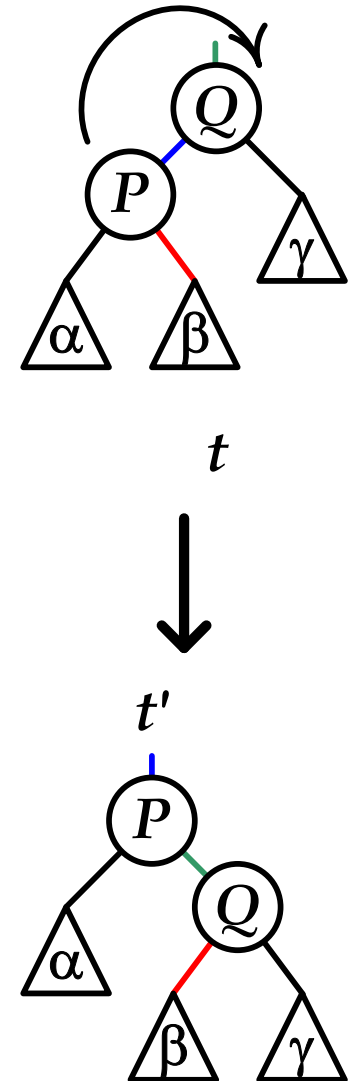
- Rebalance of AVL tree are done with simple modification to tree, known as **rotation**.
- Insertion occurs on the “outside” (i.e., **left-left** or **right-right**) is fixed by **single rotation** of the tree. **Left-left** means **left subtree** of the **left child** of the unbalanced node.
- Insertion occurs on the “inside” (i.e., **left-right** or **right-left**) is fixed by **double rotation** of the tree. **Left-right** means right **subtree** of the left **child** of the unbalance node.



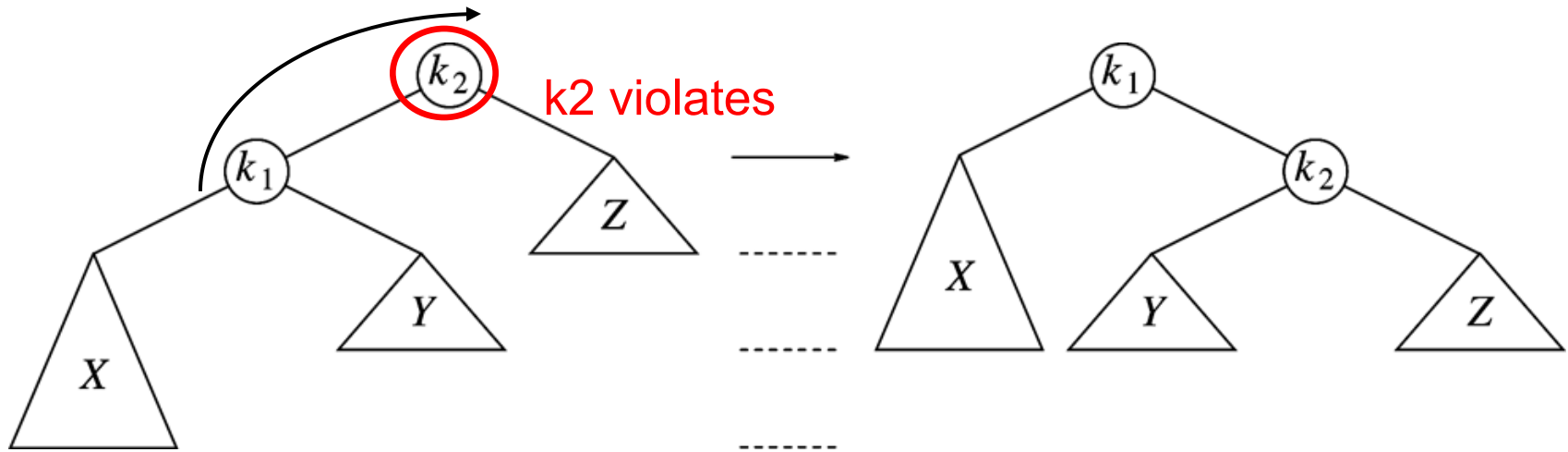
Tree Rotations

Given that P is the left child of Q , α and β are the left and right sub-trees of P , γ is the right sub-tree of Q .

- A left-to-right rotation on (sub-)tree t rooted by Q is that
 - ♦ Let Q be the right child of P ;
 - ♦ Let β be the left sub-tree of Q ;
 - ♦ Let P be the new root of tree t .
- After the rotation, α is shallowed and γ is deepened (by one level), and the depth of β is unchanged.
- It's symmetric for right-to-left rotations.
- **Tree rotations do not change a tree's in-order traversal sequence.**



Single Rotation to Fix Case 1 (left-left)



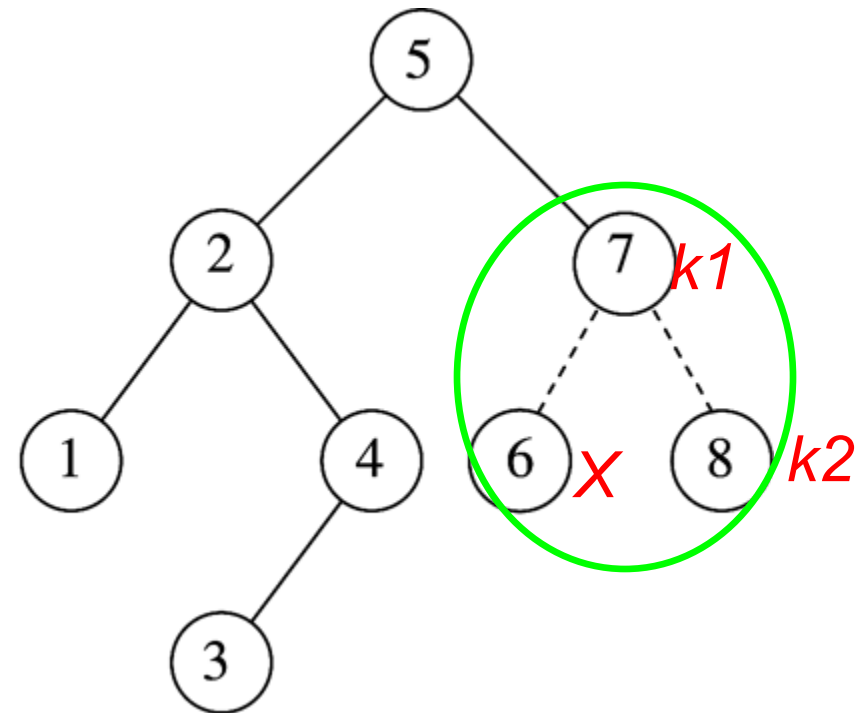
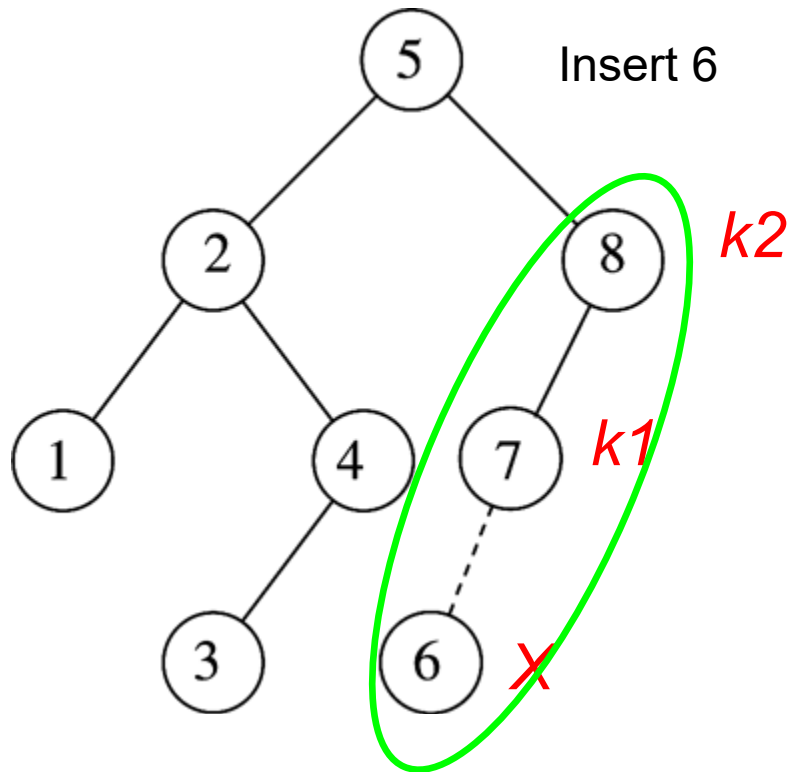
An insertion in subtree X,
AVL property violated at node k_2

Solution: single rotation

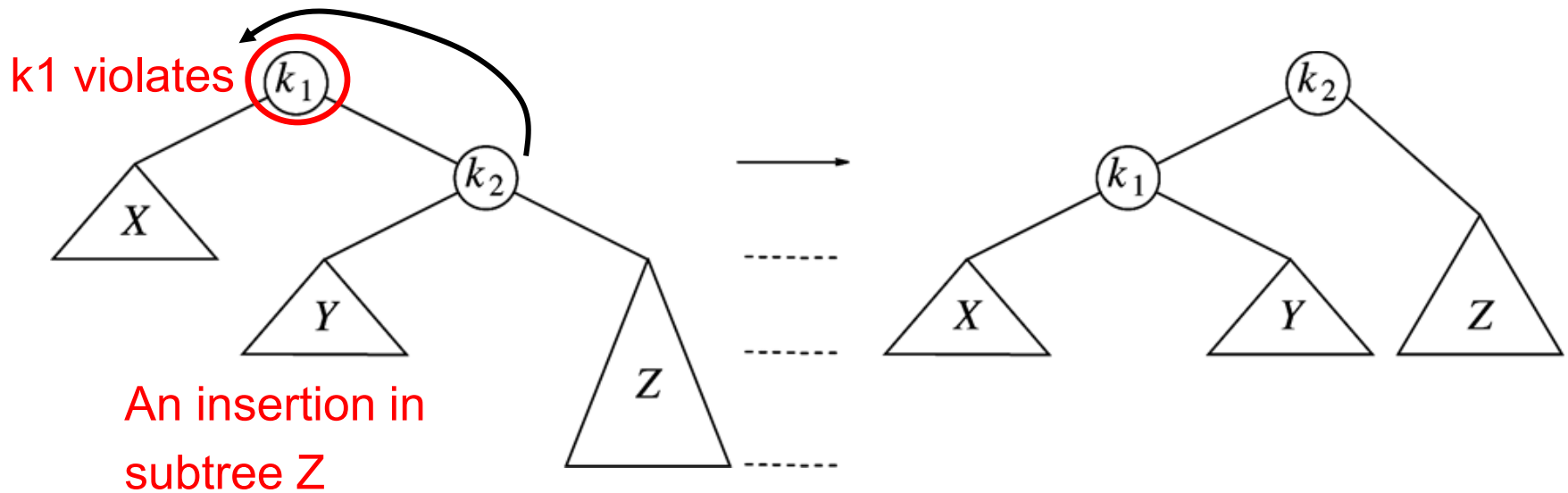
AVL-property:

- Y is at the same level as the new X;
- Y have the same height as Z.
- X is to grow. But the new height of the entire subtree is *exactly the same* as the height of the original subtree.

Single Rotation Case 1 Example



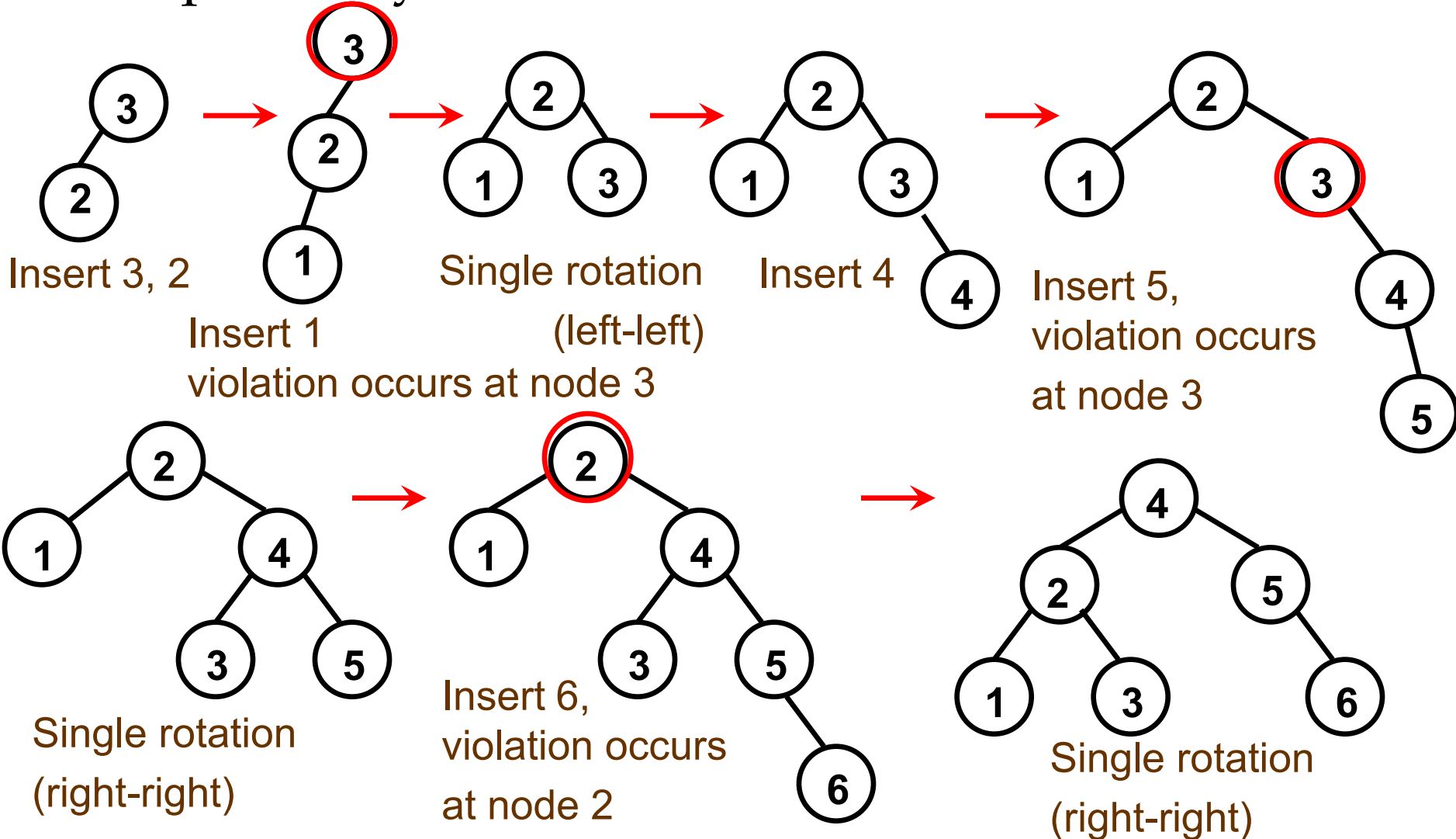
Single Rotation to Fix Case 4 (right-right)



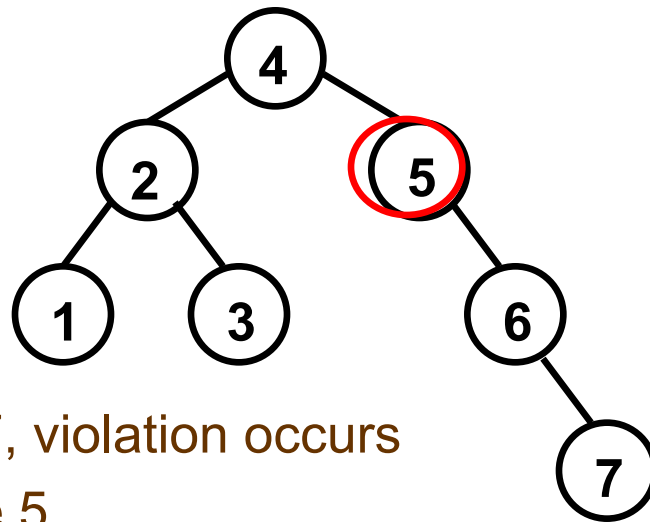
- Case 4 is a symmetric case to case 1.
- Insertion takes $O(\text{Height of AVL Tree})$ time
- Single rotation takes $O(1)$ time.

Single Rotation Example

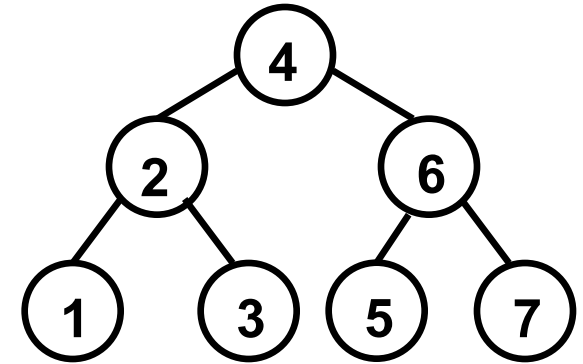
- Sequentially insert 3, 2, 1, 4, 5, 6 to an AVL Tree



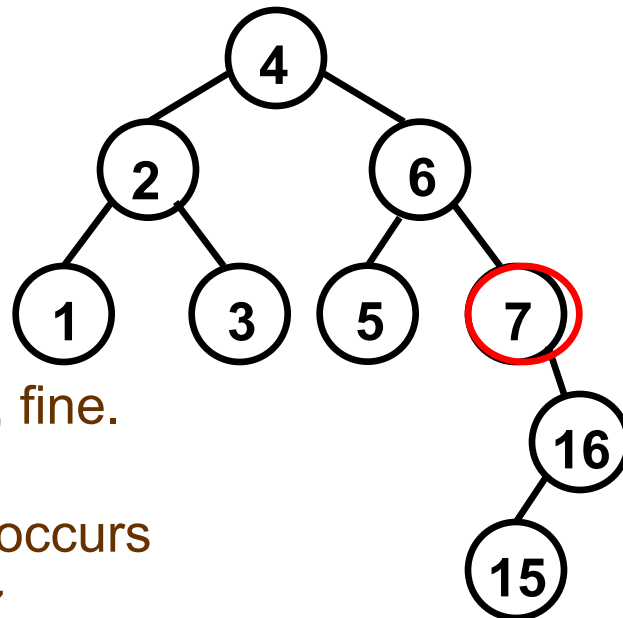
- If we continue to insert 7, 16, 15, 14, 13, 12, 11, 10, 8, 9



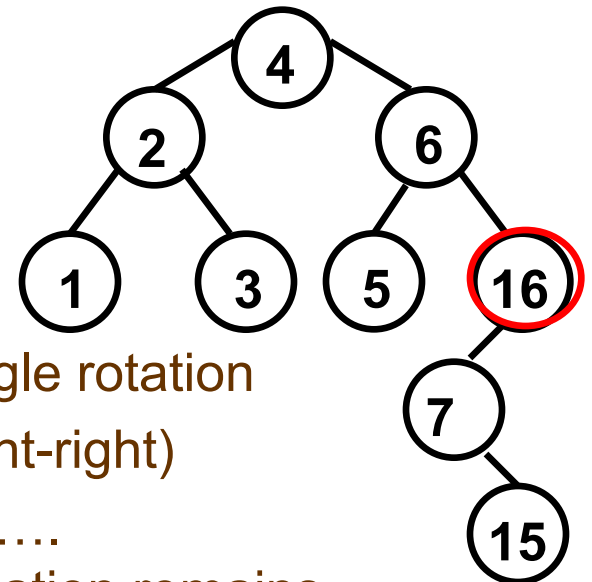
Insert 7, violation occurs
at node 5



Single rotation
(right-right)

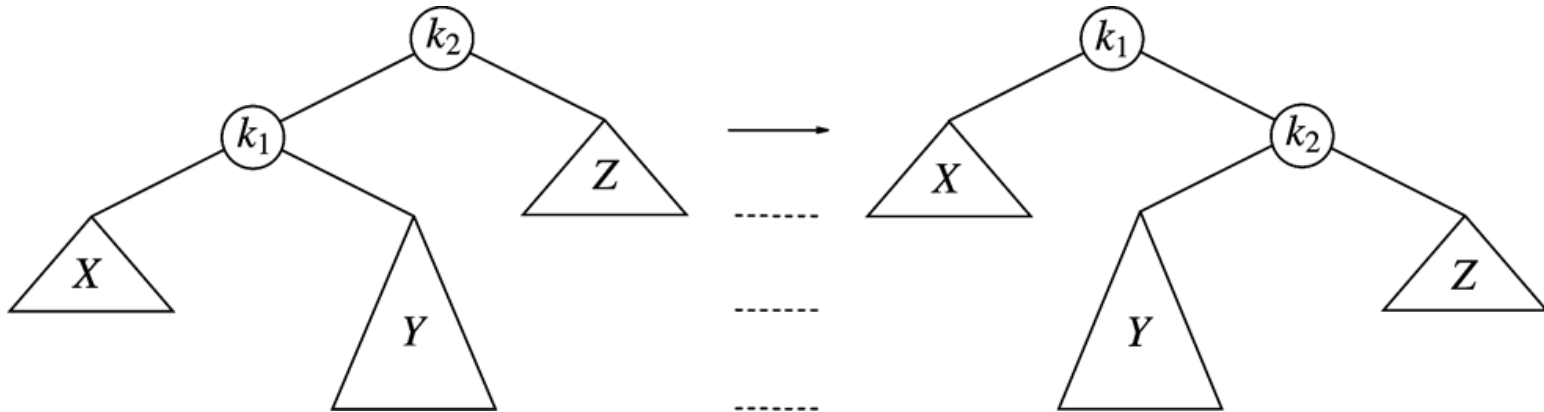


Insert 16, fine.
Insert 15
violation occurs
at node 7



Single rotation
(right-right)
But....
Violation remains

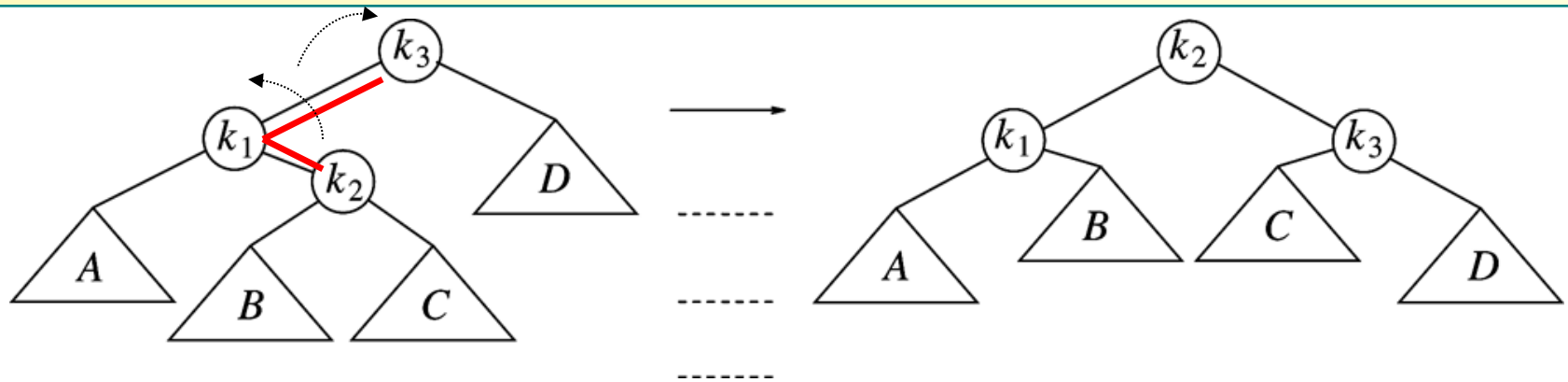
Single Rotation Fails to Fix Cases 2 & 3



Case 2: violation occurs at k_2 because of the insertion in the subtree Y The result after performing the single rotation

- Single rotation fails to fix cases 2 & 3
- Take case 2 as an example (case 3 is a symmetry to it).
 - ♦ The problem is subtree Y is too deep.
 - ♦ Single rotation doesn't make it less deep.

Double Rotation to Fix Case 2 (left-right)

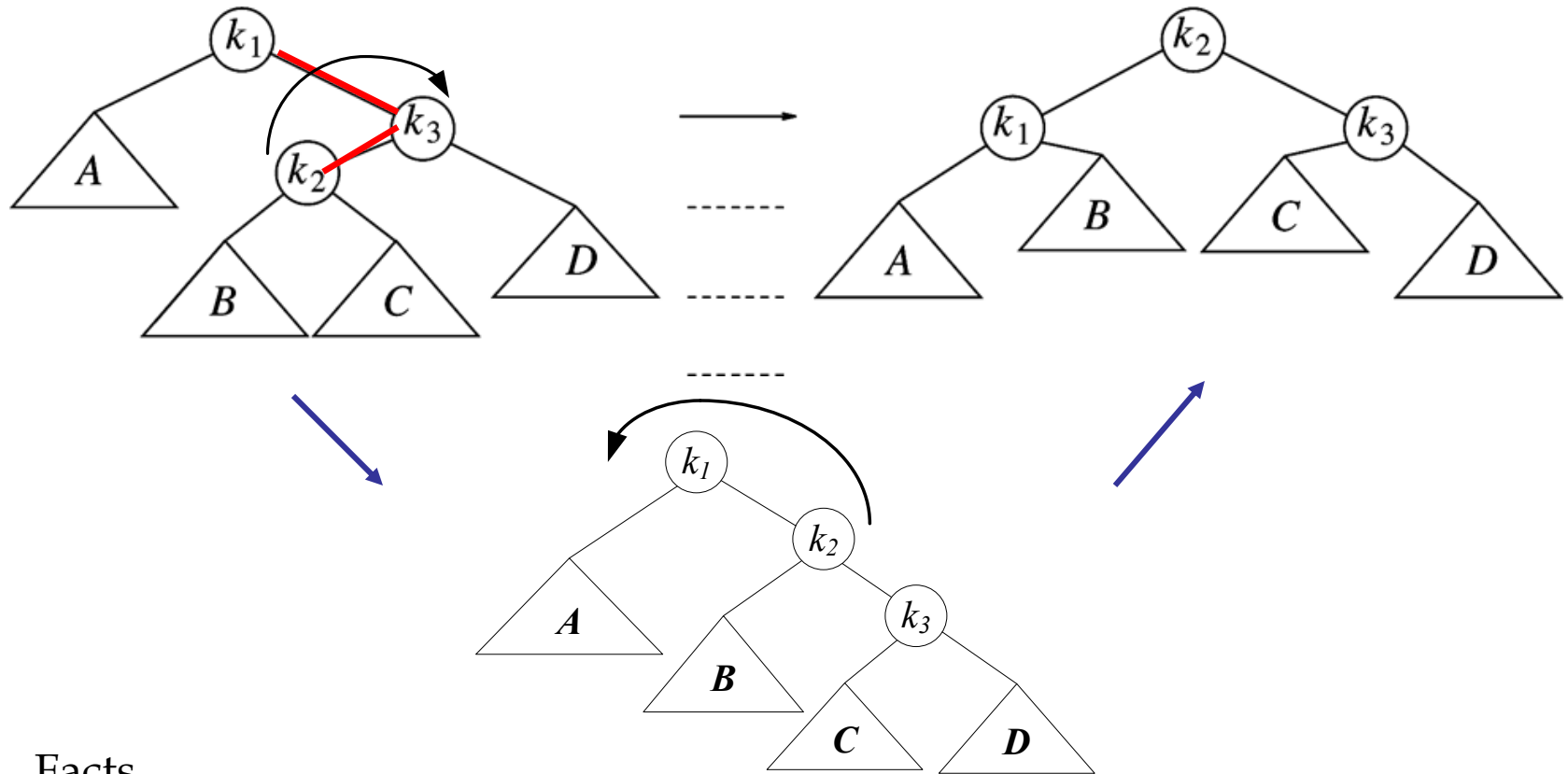


Double rotation to fix case 2

- Facts
 - ♦ The new key is inserted in the subtree B or subtree C.
 - ♦ The AVL-property is violated at k_3 .
 - ♦ k_3 - k_1 - k_2 forms a zig-zag shape (left-right).
- Solution
 - ♦ We cannot leave k_3 as the root.
 - ♦ The only alternative is to place k_2 as the new root.

Double Rotation to Fix Case 3 (right-left)

Double rotation to fix case 3



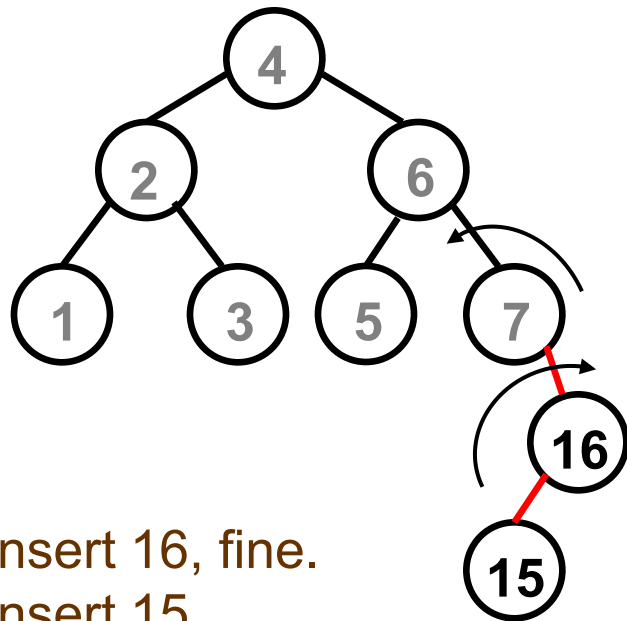
- Facts
 - ♦ The new key is inserted in the subtree B or subtree C.
 - ♦ The AVL-property is violated at k_1 .
 - ♦ k_1 - k_3 - k_2 forms a zig-zag shape (right-left).
- Case 3 is a symmetric case to case 2.

Double Rotation Example

Restart our example

We've inserted 3, 2, 1, 4, 5, 6, 7, 16

We'll insert 15, 14, 13, 12, 11, 10, 8, 9

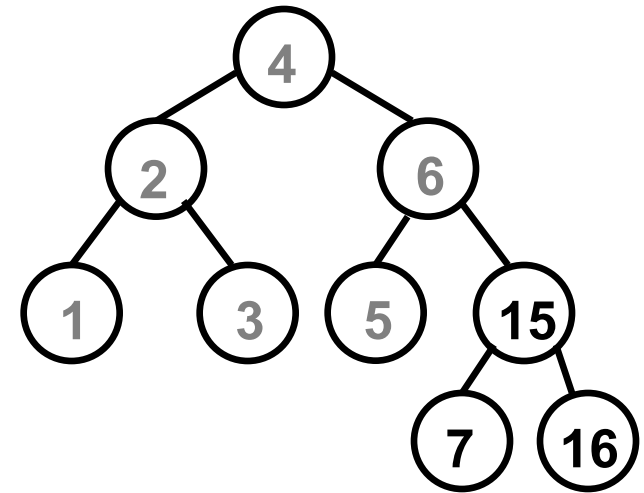


Insert 16, fine.

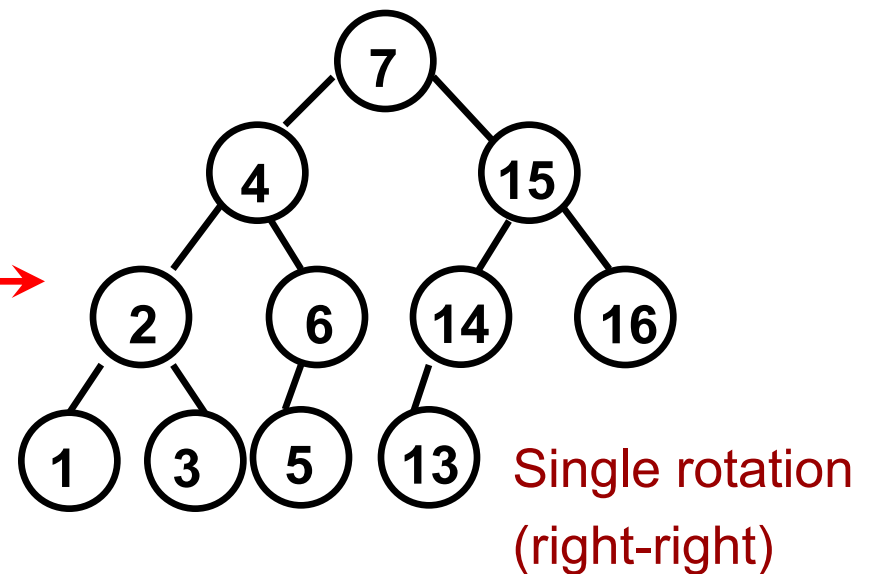
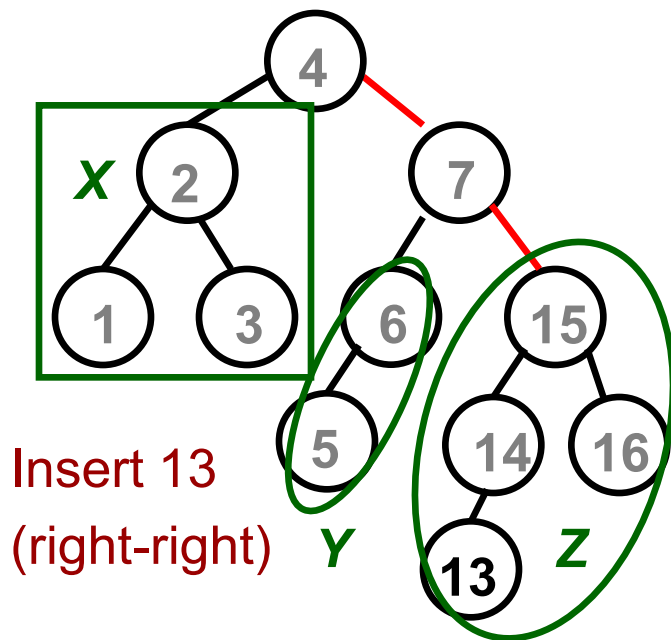
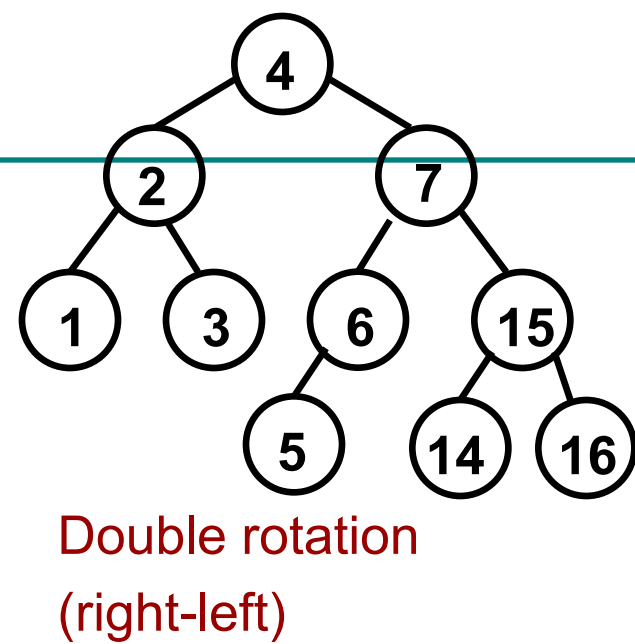
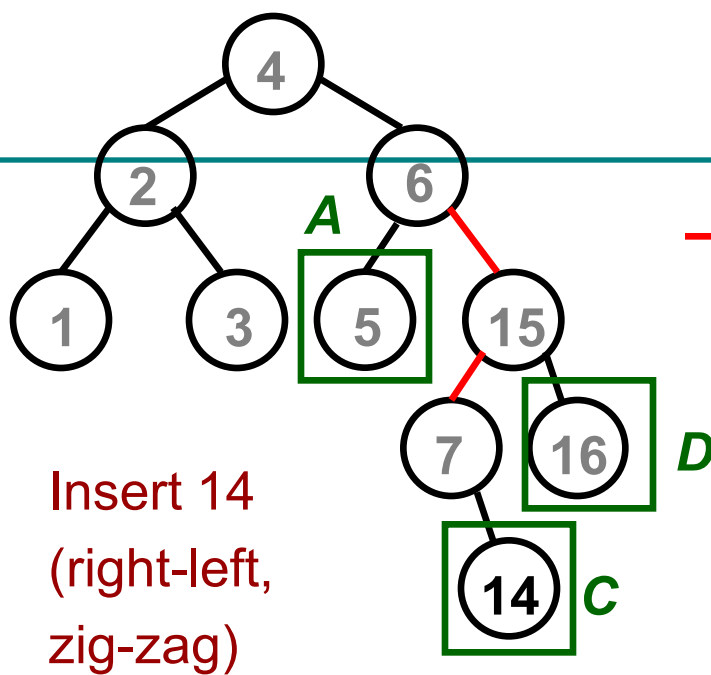
Insert 15

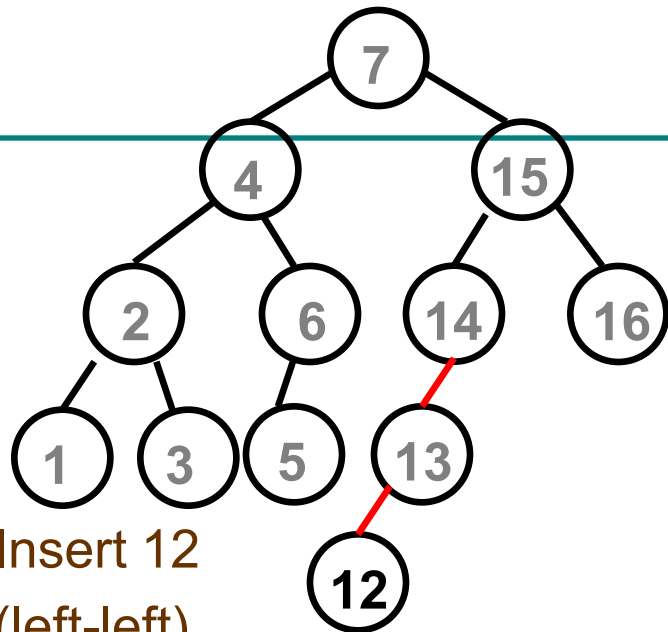
violation occurs at node 7

(right-left, zig-zag)

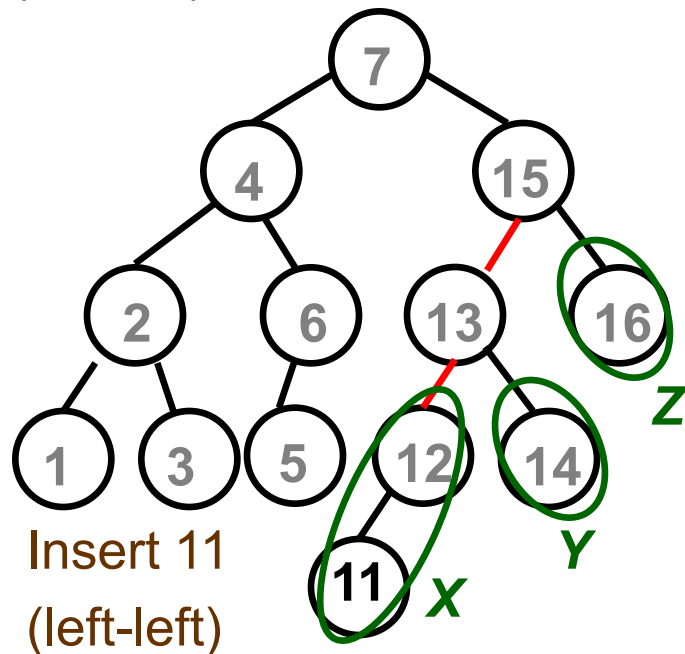


Double rotation
(right-left)

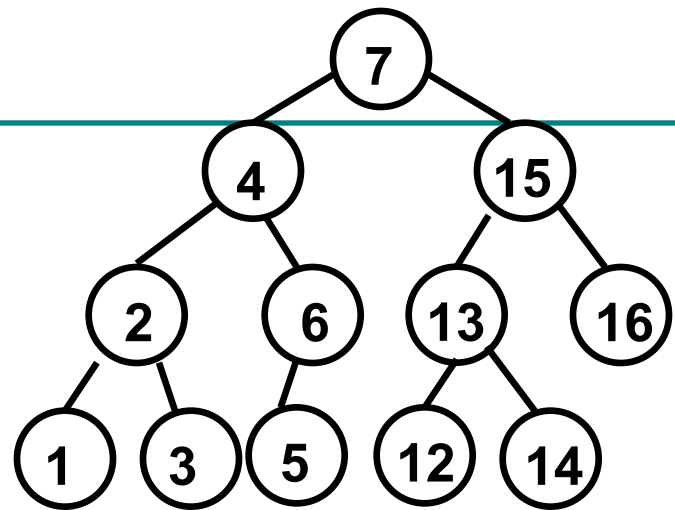




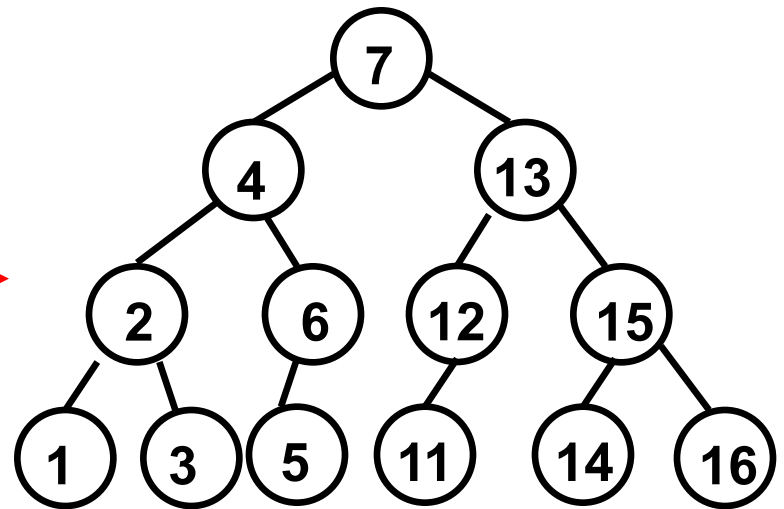
Insert 12
(left-left)



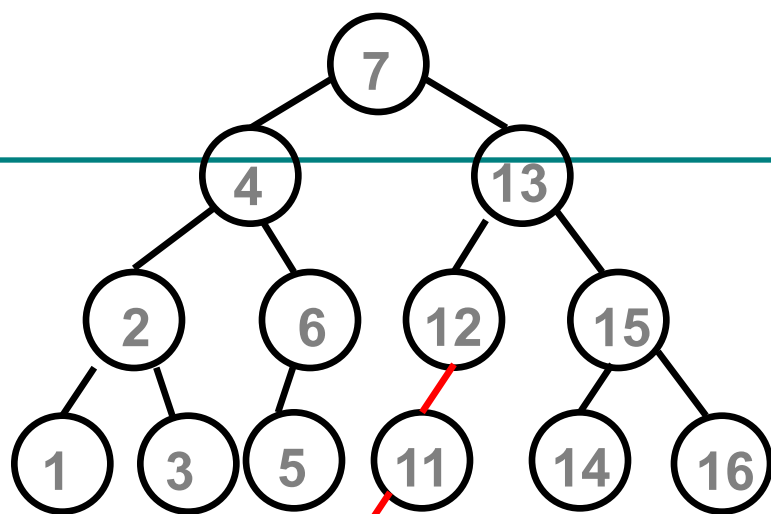
Insert 11
(left-left)



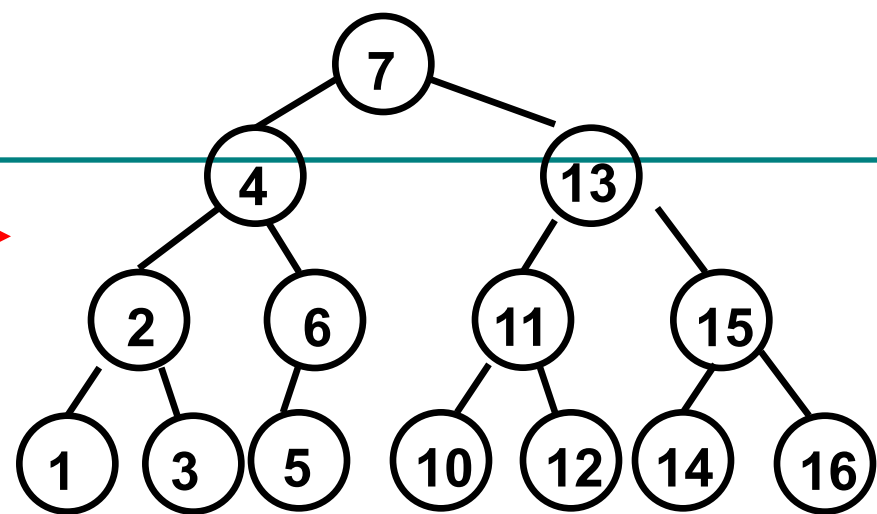
Single rotation (left-left)



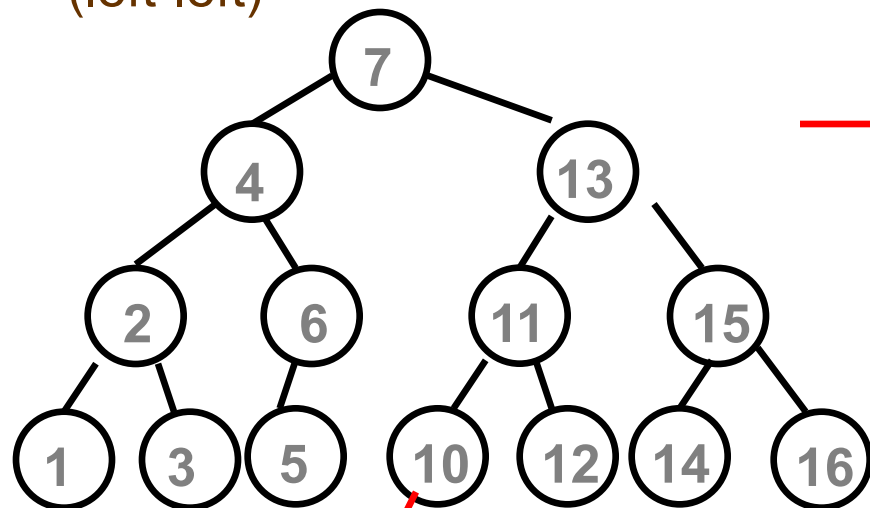
Single rotation (left-left)



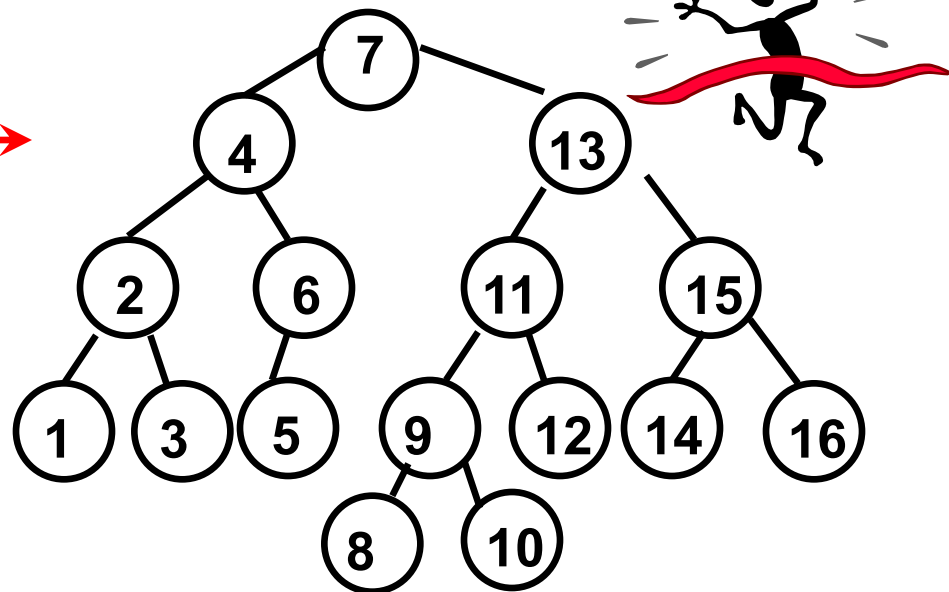
Insert 10
(left-left)



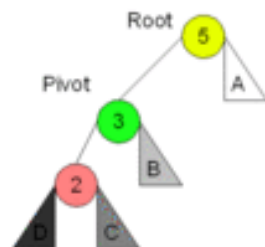
Single rotation (left-left)



Insert 8, fine
then insert 9 (left-right, zig-zag) double rotation (left-right)

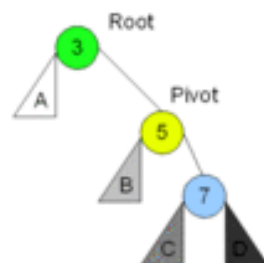


左左情况



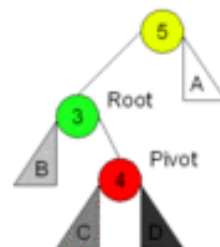
右旋

右右情况



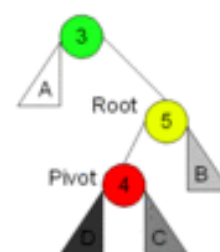
左旋

左右情况

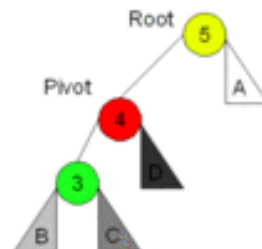


左旋

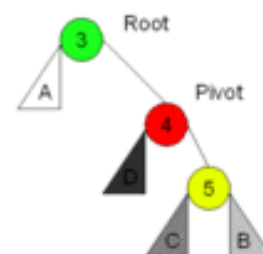
右左情况



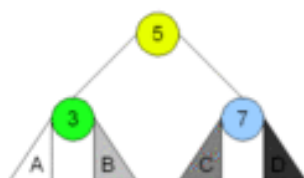
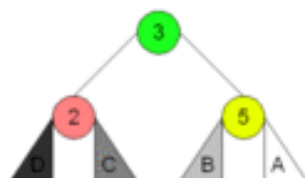
右旋



右旋



左旋



Insertion Analysis

- Insert the new key as a new leaf just as in ordinary binary search tree: $O(\log N)$
- Then trace the path from the new leaf towards the root, for each node x encountered: $O(\log N)$
 - ♦ Check height difference: $O(1)$
 - ♦ If satisfies AVL property, proceed to next node: $O(1)$
 - ♦ If not, perform a rotation: $O(1)$
- The insertion stops when
 - ♦ A single rotation is performed.
 - ♦ Or, we've checked all nodes in the path.
- Time complexity for insertion $O(\log N)$.

Implementation

- Node declaration for AVL trees

```
struct AvlNode;  
typedef struct AvlNode *Position;  
typedef struct AvlNode *AvlTree;  
  
struct AvlNode  
{  
    ElementType Element;  
    AvlTree Left;  
    AvlTree Right;  
    int Height;  
};
```

- Return the height of a node in the AVL tree

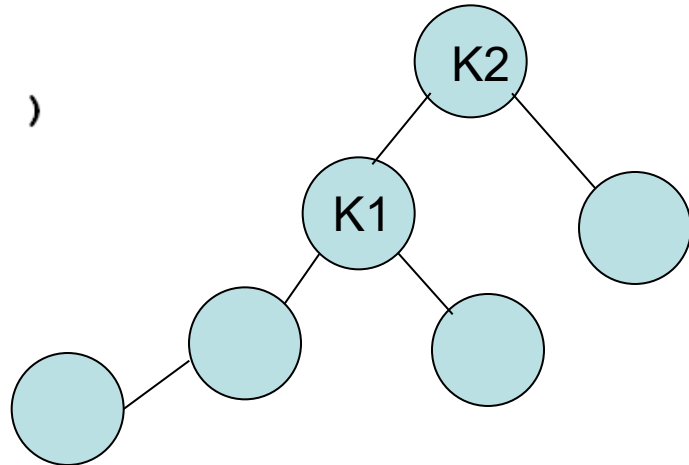
```
static int  
Height( Position P )  
{  
    if( P == NULL )  
        return -1;  
    else  
        return P->Height;  
}
```

Implementation (cont'd)

- Single Rotation -- Left-Left

```
/* This function can be called only if K2 has a left child */  
/* Perform a rotate between a node (K2) and its left child */  
/* Update heights, then return new root */
```

```
static Position  
SingleRotateWithLeft( Position K2 )  
{  
    Position K1;  
  
    K1 = K2->Left;  
    K2->Left = K1->Right;  
    K1->Right = K2;  
  
    K2->Height = Max( Height( K2->Left ), Height( K2->Right ) ) + 1;  
    K1->Height = Max( Height( K1->Left ), K2->Height ) + 1;  
  
    return K1; /* New root */  
}
```



Implementation (cont'd)

- Single rotation -- Right-Right

```
/* This function can be called only if K1 has a right child */  
/* Perform a rotate between a node (K1) and its right child */  
/* Update heights, then return new root */
```

```
static Position  
SingleRotateWithRight( Position K1 )  
{  
    Position K2;  
  
    K2 = K1->Right;  
    K1->Right = K2->Left;  
    K2->Left = K1;  
  
    K1->Height = Max( Height( K1->Left ), Height( K1->Right ) ) + 1;  
    K2->Height = Max( Height( K2->Right ), K1->Height ) + 1;  
  
    return K2; /* New root */  
}
```

Implementation (cont'd)

- Double rotation -- Left-Right

```
/* This function can be called only if K3 has a left */  
/* child and K3's left child has a right child */  
/* Do the left-right double rotation */  
/* Update heights, then return new root */
```

```
static Position
```

```
DoubleRotateWithLeft( Position K3 )
```

```
{
```

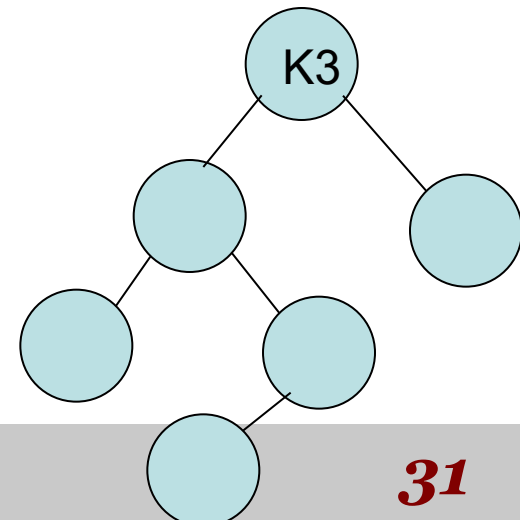
```
    /* Rotate between K1 and K2 */
```

```
    K3->Left = SingleRotateWithRight( K3->Left );
```

```
    /* Rotate between K3 and K2 */
```

```
    return SingleRotateWithLeft( K3 );
```

```
}
```



Implementation (cont'd)

- Double rotation -- Right-Left

```
/* This function can be called only if K1 has a right */  
/* child and K1's right child has a left child */  
/* Do the right-left double rotation */  
/* Update heights, then return new root */
```

```
static Position
```

```
DoubleRotateWithRight( Position K1 )
```

```
{
```

```
    /* Rotate between K3 and K2 */
```

```
    K1->Right = SingleRotateWithLeft( K1->Right );
```

```
    /* Rotate between K1 and K2 */
```

```
    return SingleRotateWithRight( K1 );
```

```
}
```


Insertion Algorithm

- First, insert the new key as a new leaf just as in ordinary binary search tree.
- Then trace the path **from the new leaf towards the root**. For each node x encountered, check if heights of $\text{left}(x)$ and $\text{right}(x)$ differ by at most 1.
 - ♦ If yes, proceed to $\text{parent}(x)$.
 - ♦ If not, re-structure by performing either a single rotation or a double rotation.
- Note: once we perform a rotation at a node x , we won't need to perform any rotation at any ancestor of x . Rotation (either single or double) needs to perform only once.

Insertion Algorithm Implementation

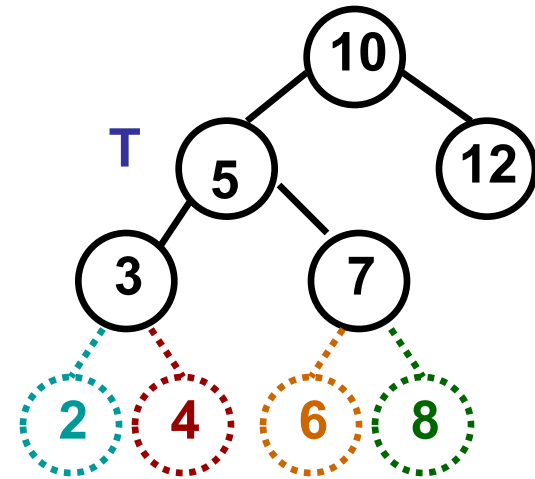
```
AvlTree
Insert( ElementType X, AvlTree T )
{
    if( T == NULL )
    {
        /* Create and return a one-node tree */
        T = malloc( sizeof( struct AvlNode ) );
        if( T == NULL )
            FatalError( "Out of space!!!" );
        else
        {
            T->Element = X; T->Height = 0;
            T->Left = T->Right = NULL;
        }
    }

    .....
}
```

Insertion Algorithm Implementation

```
else
if( X < T->Element )
{
    T->Left = Insert( X, T->Left );
    if( Height( T->Left ) - Height( T->Right ) == 2 )
        if( X < T->Left->Element )
            T = SingleRotateWithLeft( T );
        else
            T = DoubleRotateWithLeft( T );
}
else
if( X > T->Element )
{
    T->Right = Insert( X, T->Right );
    if( Height( T->Right ) - Height( T->Left ) == 2 )
        if( X > T->Right->Element )
            T = SingleRotateWithRight( T );
        else
            T = DoubleRotateWithRight( T );
}
/* Else X is in the tree already; we'll do nothing */

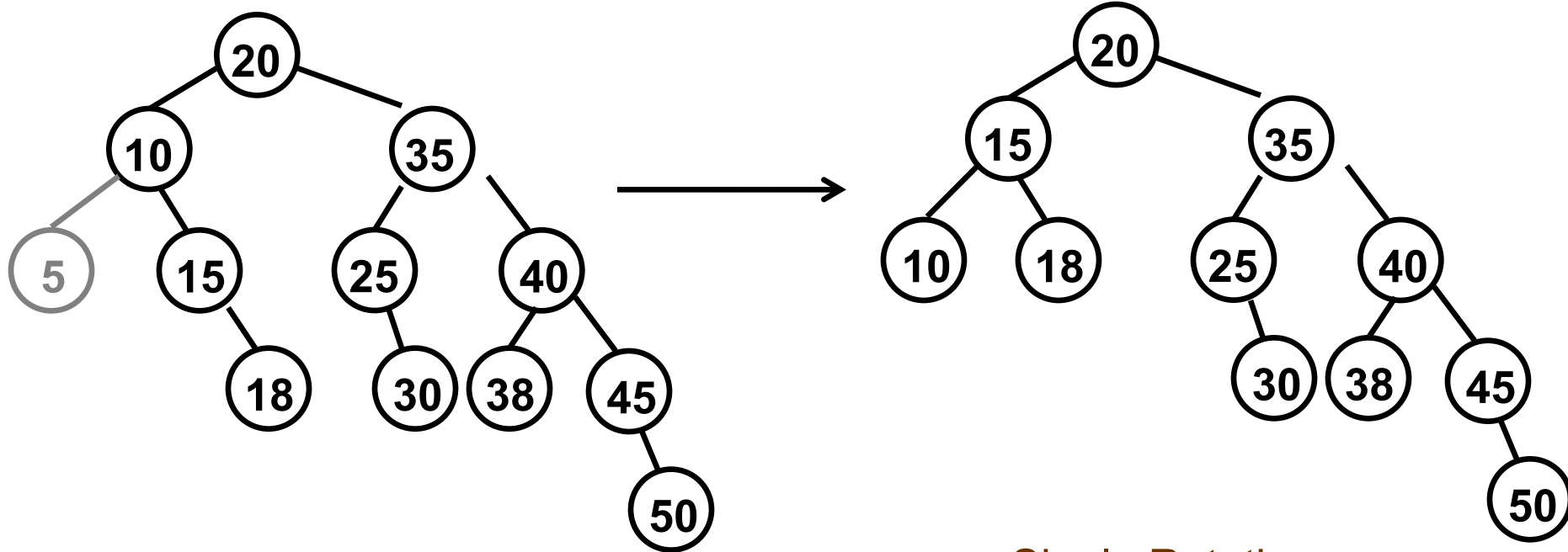
T->Height = Max( Height( T->Left ), Height( T->Right ) ) + 1;
return T;
}
```



How about Deletion from AVL Trees?

- Deletion in AVL trees is somewhat more complicated than insertion.
- Firstly, delete a node x as in ordinary binary search tree
 - ♦ Note that the last node deleted is **a leaf** (case I in BST deletion) or **a node with one child** (case II in BST deletion).
- Then trace the path from **the last node deleted towards the root** (deletion path).
 - ♦ For each node x encountered, check if heights of $\text{left}(x)$ and $\text{right}(x)$ differ by at most 1.
 - If yes, proceed to $\text{parent}(x)$.
 - If no, perform an appropriate rotation at x .
 - ♦ Continue to trace the path until we reach the root.
- Unlike insertion, **more than one node** may need rotation.

Deletion Example

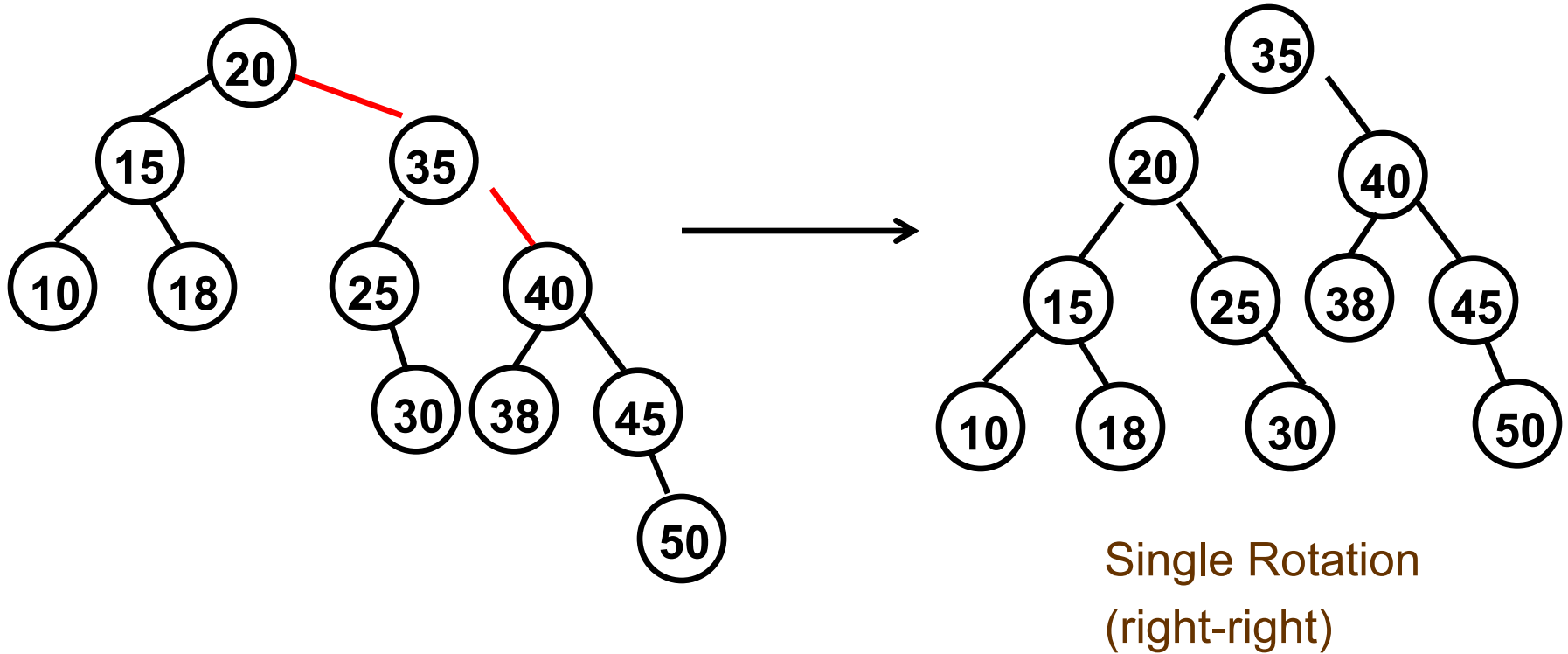


Delete 5

Node 10 is unbalanced
(right-right)

Single Rotation

Deletion Example (cont'd)



Conclusion

- AVL tree
 - ♦ a *binary search tree* with a balanced condition in which for *every* node in the tree, the height of the left and right subtrees differ by at most 1.
- Keep the balance
 - ♦ Single rotation
 - ♦ Double rotation
- Implementation
 - ♦ Insertion
 - ♦ Deletion

Assignment 09

- Textbook, 4.16
- Submit it to Moodle